Database Engineering

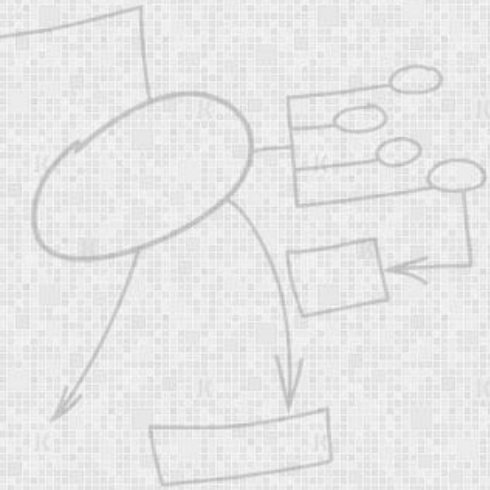
Lecture 7: Structured Query Language (SQL)

By

Dr Wasi Haider Butt

SQL Overview

- Structured Query Language
- The standard for relational database management systems (RDBMS)

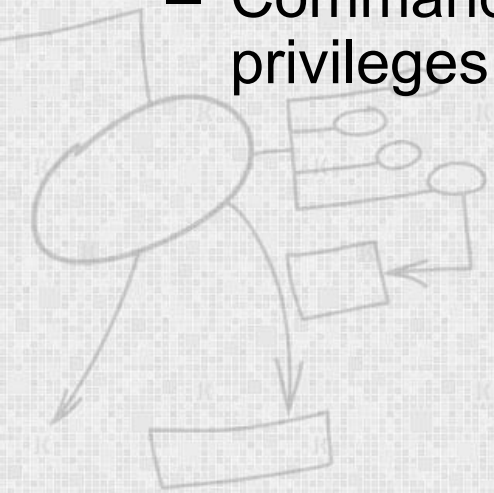


History of SQL

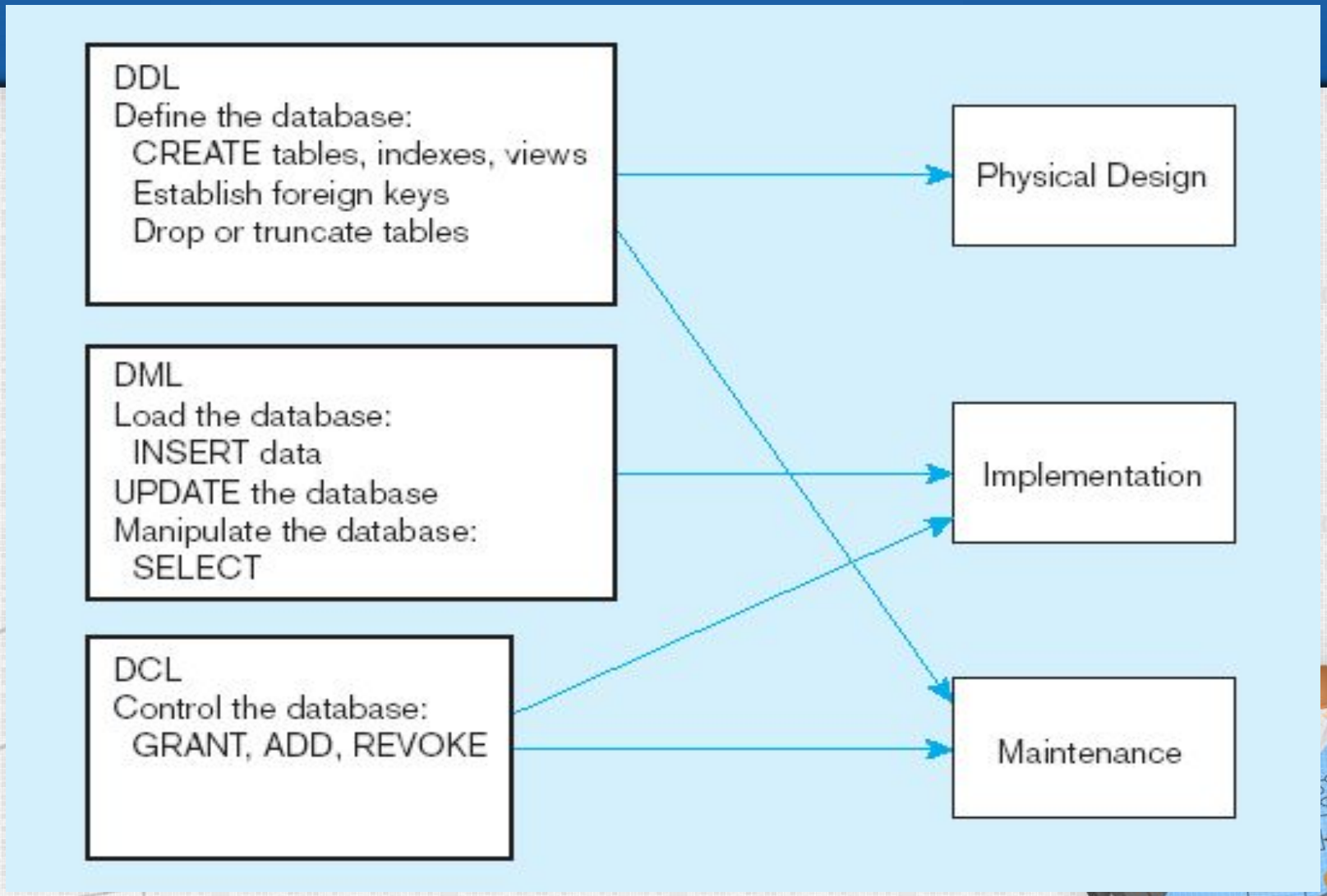
- 1970–E. Codd develops relational database concept
- 1974-1979–System R with Sequel (later SQL) created at IBM Research Lab
- 1979–Oracle markets first relational DB with SQL
- 1986–ANSI SQL standard released
- 1989, 1992, 1999, 2003–Major ANSI standard updates
- Current–SQL is supported by most major database vendors

SQL Environment

- Data Definition Language (DDL)
  - Commands that define a database, including creating, altering, and dropping tables and establishing constraints
- Data Manipulation Language (DML)
  - Commands that maintain and query a database
- Data Control Language (DCL)
  - Commands that control a database, including administering privileges



DDL, DML, DCL, and the database development process



SQL Database Definition

- Data Definition Language (DDL)
- Major CREATE statements:
  - CREATE SCHEMA–create database
  - CREATE TABLE–defines a table and its columns
  - CREATE VIEW–defines a logical table from one or more views

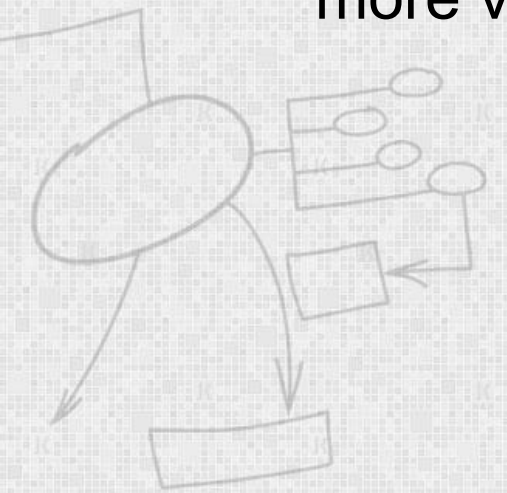
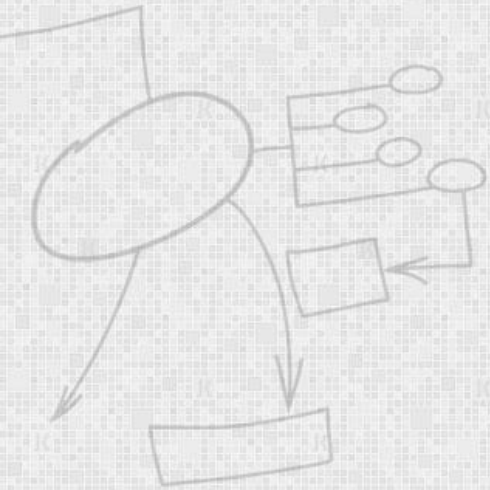


Table Creation

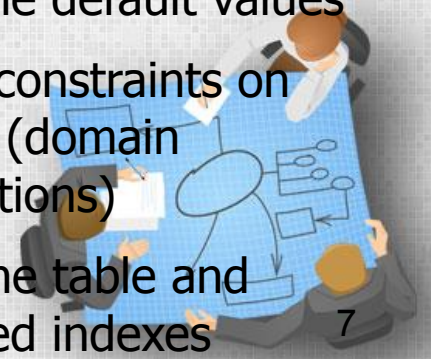
General syntax for CREATE TABLE

```
CREATE TABLE tablename
( {column definition   [table constraint] } . , . .
```

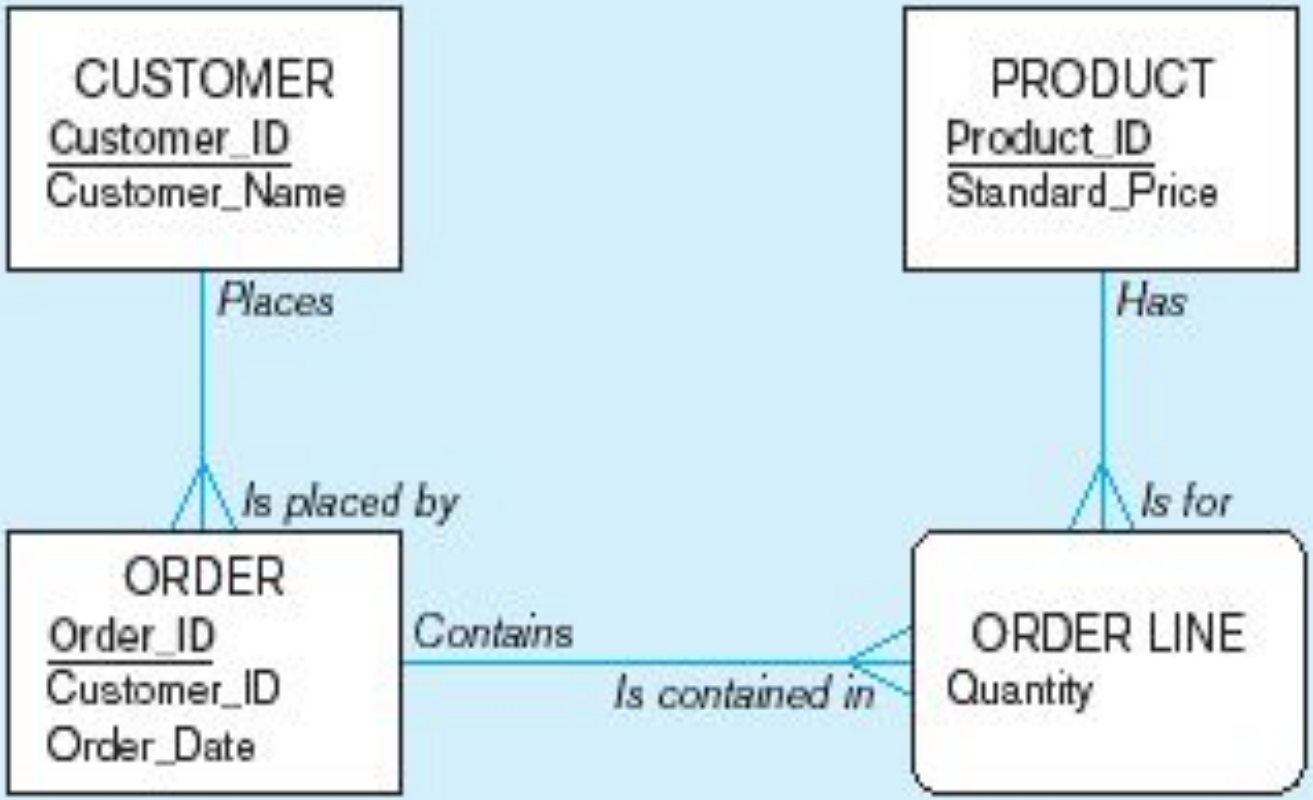


Steps in table creation:

1. Identify data types for attributes
2. Identify columns that can and cannot be null
3. Identify columns that must be unique (candidate keys)
4. Identify primary key–foreign key mates
5. Determine default values
6. Identify constraints on columns (domain specifications)
7. Create the table and associated indexes



Example



database definition commands for Pine Valley Furniture

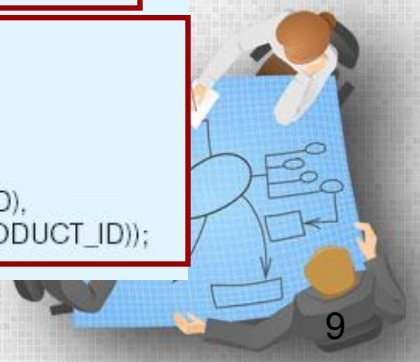
```
CREATE TABLE CUSTOMER_T
              (CUSTOMER_ID                NUMBER(11, 0) NOT NULL,
               CUSTOMER_NAME              VARCHAR2(25)  NOT NULL,
               CUSTOMER_ADDRESS           VARCHAR2(30),
               CITY                       VARCHAR2(20),
               STATE                      VARCHAR2(2),
               POSTAL_CODE                VARCHAR2(9),
CONSTRAINT CUSTOMER_PK PRIMARY KEY (CUSTOMER_ID));
```

Overall table definitions

```
CREATE TABLE ORDER_T
              (ORDER_ID                   NUMBER(11, 0) NOT NULL,
               ORDER_DATE                 DATE DEFAULT SYSDATE,
               CUSTOMER_ID                NUMBER(11, 0),
CONSTRAINT ORDER_PK PRIMARY KEY (ORDER_ID),
CONSTRAINT ORDER_FK FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER_T(CUSTOMER_ID));
```

```
CREATE TABLE PRODUCT_T
              (PRODUCT_ID                 INTEGER        NOT NULL,
               PRODUCT_DESCRIPTION        VARCHAR2(50),
               PRODUCT_FINISH             VARCHAR2(20)
                          CHECK (PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'White Ash',
                                 'Red Oak', 'Natural Oak', 'Walnut')),
               STANDARD_PRICE             DECIMAL(6,2),
               PRODUCT_LINE_ID            INTEGER,
CONSTRAINT PRODUCT_PK PRIMARY KEY (PRODUCT_ID));
```

```
CREATE TABLE ORDER_LINE_T
              (ORDER_ID                   NUMBER(11,0)  NOT NULL,
               PRODUCT_ID                 NUMBER(11,0)  NOT NULL,
               ORDERED_QUANTITY           NUMBER(11,0),
CONSTRAINT ORDER_LINE_PK PRIMARY KEY (ORDER_ID, PRODUCT_ID),
CONSTRAINT ORDER_LINE_FK1 FOREIGN KEY(ORDER_ID) REFERENCES ORDER_T(ORDER_ID),
CONSTRAINT ORDER_LINE_FK2 FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT_T(PRODUCT_ID));
```



Defining attributes and their data types

```
CREATE TABLE PRODUCT_T
        (PRODUCT_ID              INTEGER       NOT NULL,
         PRODUCT_DESCRIPTION     VARCHAR2(50),
         PRODUCT_FINISH          VARCHAR2(20)
                    CHECK (PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'White Ash',
                                'Red Oak', 'Natural Oak', 'Walnut')),
         STANDARD_PRICE          DECIMAL(6,2),
         PRODUCT_LINE_ID         INTEGER,
CONSTRAINT PRODUCT_PK PRIMARY KEY (PRODUCT_ID));
```

Non-nullable specification

```
CREATE TABLE PRODUCT_T
    (PRODUCT_ID             INTEGER     NOT NULL,
     PRODUCT_DESCRIPTION    VARCHAR2(50),
     PRODUCT_FINISH         VARCHAR2(20)
                CHECK (PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'White Ash',
                       'Red Oak', 'Natural Oak', 'Walnut')),
     STANDARD_PRICE         DECIMAL(6,2),
     PRODUCT_LINE_ID        INTEGER,
CONSTRAINT PRODUCT_PK PRIMARY KEY (PRODUCT_ID));
```

Identifying primary key

Primary keys can never have NULL values

```
CREATE TABLE ORDER_LINE_T
        (ORDER_ID              NUMBER(11,0) NOT NULL,
         PRODUCT_ID            NUMBER(11,0) NOT NULL,
         ORDERED_QUANTITY      NUMBER(11,0),
CONSTRAINT ORDER_LINE_PK PRIMARY KEY (ORDER_ID, PRODUCT_ID),
CONSTRAINT ORDER_LINE_FK1 FOREIGN KEY(ORDER_ID) REFERENCES ORDER_T(ORDER_ID),
CONSTRAINT ORDER_LINE_FK2 FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT_T(PRODUCT_ID));
```

Non-nullable specifications

Primary key

Some primary keys are composite– composed of multiple attributes

Controlling the values in attributes

```
CREATE TABLE ORDER_T
        (ORDER_ID           NUMBER(11, 0) NOT NULL,
         ORDER_DATE         DATE          DEFAULT SYSDATE,
         CUSTOMER_ID        NUMBER(11, 0),
CONSTRAINT ORDER_PK PRIMARY KEY (ORDER_ID),
CONSTRAINT ORDER_FK FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER_T(CUSTOMER_ID));

CREATE TABLE PRODUCT_T
        (PRODUCT_ID         INTEGER       NOT NULL,
         PRODUCT_DESCRIPTION VARCHAR2(50),
         PRODUCT_FINISH     VARCHAR2(20)
                CHECK (PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'White Ash',
                        'Red Oak', 'Natural Oak', 'Walnut')),
         STANDARD_PRICE     DECIMAL(6,2),
         PRODUCT_LINE_ID    INTEGER,
```

Default value

Domain constraint

Identifying foreign keys and establishing relationships

```
CREATE TABLE CUSTOMER_T
        (CUSTOMER_ID            NUMBER(11, 0) NOT NULL,
         CUSTOMER_NAME          VARCHAR2(25) NOT NULL,
         CUSTOMER_ADDRESS       VARCHAR2(30),
         CITY                   VARCHAR2(20),
         STATE                  VARCHAR2(2),
         POSTAL_CODE            VARCHAR2(9),
CONSTRAINT CUSTOMER_PK PRIMARY KEY (CUSTOMER_ID));

CREATE TABLE ORDER_T
        (ORDER_ID               NUMBER(11, 0) NOT NULL,
         ORDER_DATE             DATE          DEFAULT SYSDATE,
         CUSTOMER_ID            NUMBER(11, 0),
CONSTRAINT ORDER_PK PRIMARY KEY (ORDER_ID),
CONSTRAINT ORDER_FK FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER_T(CUSTOMER_ID));
```

Primary key of parent table

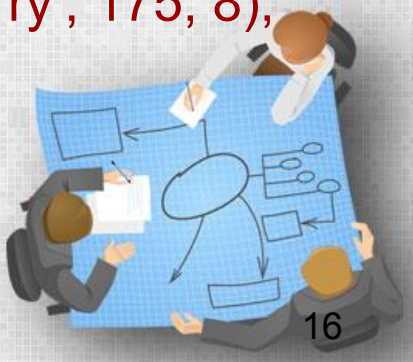
Foreign key of dependent table

Changing and Removing Tables

- ALTER TABLE statement allows you to change column specifications:
  - ALTER TABLE CUSTOMER_T ADD (cellno VARCHAR(2))
  - ALTER TABLE CUSTOMER_T DROP (cellno)
- DROP TABLE statement allows you to remove tables from your schema:
  - DROP TABLE CUSTOMER_T

Insert Statement

- Adds data to a table
- Inserting into a table
  - INSERT INTO CUSTOMER_T VALUES (001, ‘Contemporary Casuals’, ‘1355 S. Himes Blvd.’, ‘Gainesville’, ‘FL’, 32601);
- Inserting a record that has some null attributes requires identifying the fields that actually get data
  - INSERT INTO PRODUCT_T (PRODUCT_ID, PRODUCT_DESCRIPTION,PRODUCT_FINISH, STANDARD_PRICE, PRODUCT_ON_HAND) VALUES (1, ‘End Table’, ‘Cherry’, 175, 8);



Delete Statement

- Removes rows from a table
- Delete certain rows
  - DELETE FROM CUSTOMER_T WHERE STATE = ‘HI’;
- Delete all rows
  - DELETE FROM CUSTOMER_T;

Update Statement

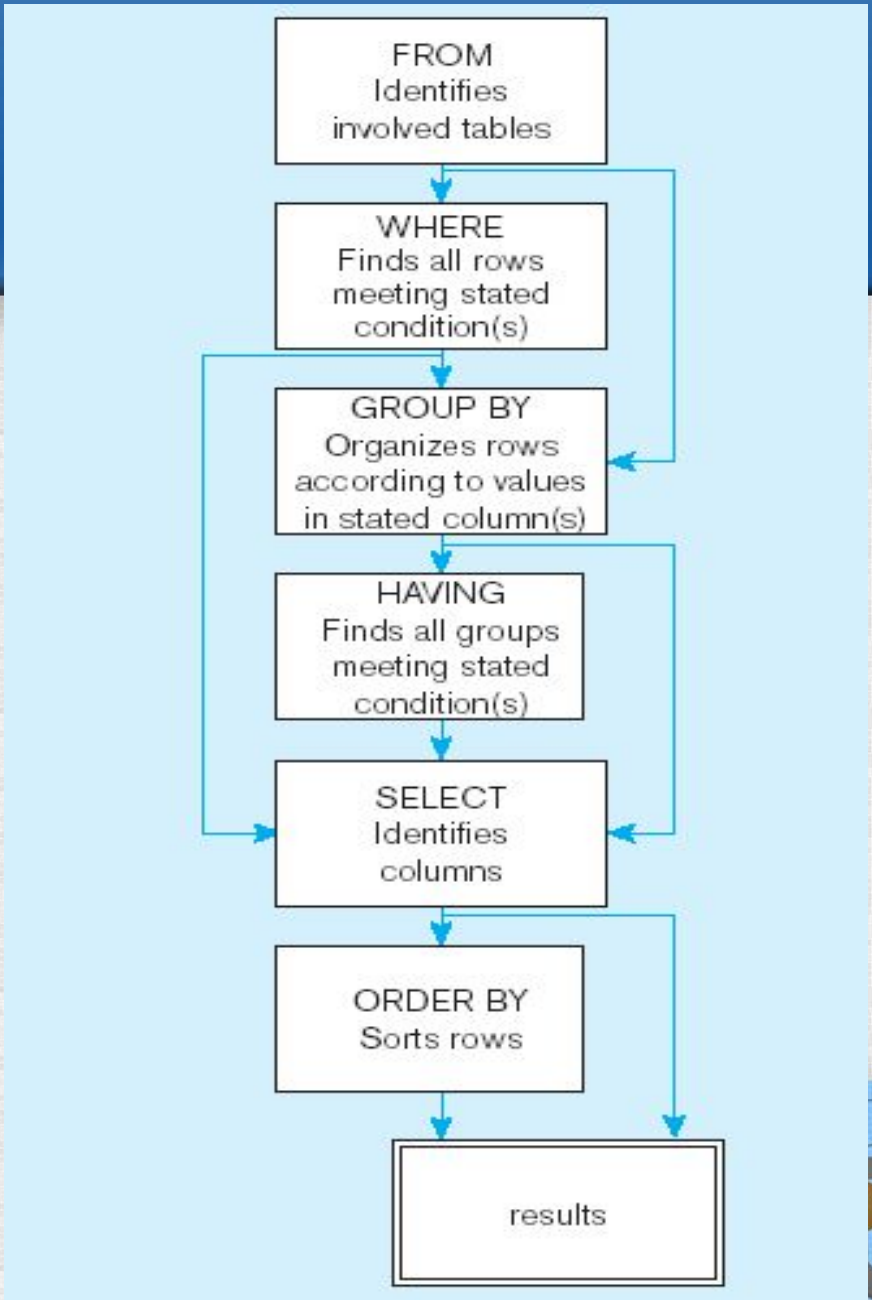
- Modifies data in existing rows

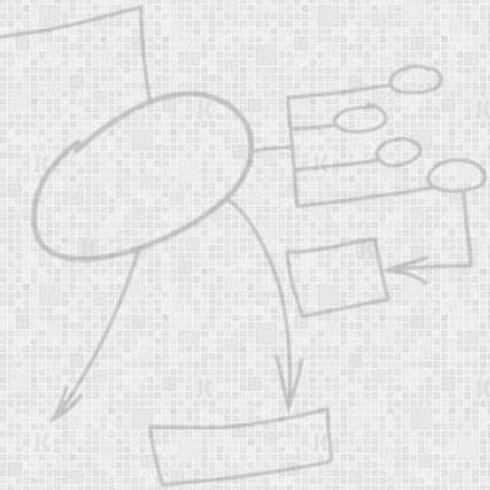
- UPDATE PRODUCT_T SET UNIT_PRICE = 775 WHERE PRODUCT_ID = 7;

SELECT Statement

- Used for queries on single or multiple tables
- Clauses of the SELECT statement:
  - SELECT
    - List the columns (and expressions) that should be returned from the query
  - FROM
    - Indicate the table(s) or view(s) from which data will be obtained
  - WHERE
    - Indicate the conditions under which a row will be included in the result
  - GROUP BY
    - Indicate categorization of results
  - HAVING
    - Indicate the conditions under which a category (group) will be included
  - ORDER BY
    - Sorts the result according to specified criteria

SQL statement processing order

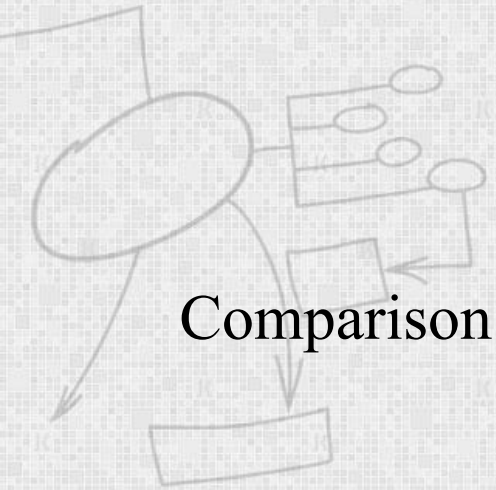




SELECT Example

- Find products with standard price less than $275

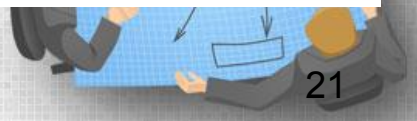
```
SELECT PRODUCT_NAME, STANDARD_PRICE
   FROM PRODUCT_V
   WHERE STANDARD_PRICE < 275;
```



Comparison Operators in SQL

Table 7-3 Comparison Operators in SQL

Operator	*Meaning*
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to
!=	Not equal to



SELECT Example Using Alias

- Alias is an alternative column or table name

```
SELECT CUST.CUSTOMER AS NAME,
    CUST.CUSTOMER_ADDRESS
FROM CUSTOMER_V CUST
    WHERE NAME = 'Home Furnishings';
```

SELECT Example Using a Function

- Using the COUNT ***aggregate function*** to find totals

```
SELECT COUNT(*) FROM ORDER_LINE_V
    WHERE ORDER_ID = 1004;
```

Note: with aggregate functions you can't have single-valued columns included in the SELECT clause

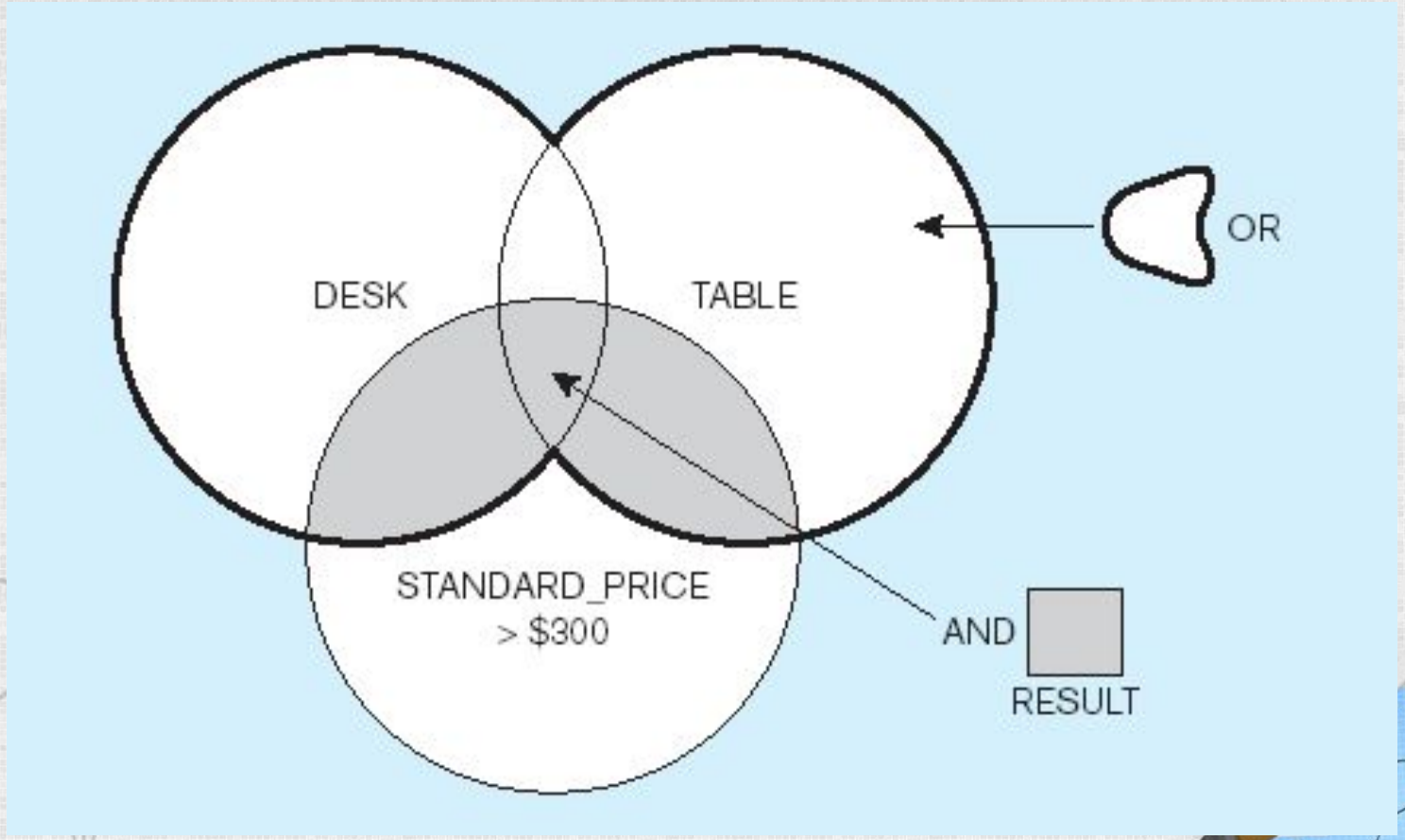
SELECT Example–Boolean Operators

- AND, OR, and NOT Operators for customizing conditions in WHERE clause

```
SELECT PRODUCT_DESCRIPTION, PRODUCT_FINISH,
   STANDARD_PRICE
FROM PRODUCT_V
   WHERE (PRODUCT_DESCRIPTION LIKE '%Desk'
   OR PRODUCT_DESCRIPTION LIKE '%Table')
   AND UNIT_PRICE > 300;
```

Note: the LIKE operator allows you to compare strings using wildcards. For example, the % wildcard in '%Desk' indicates that all strings that have any number of characters preceding the word "Desk" will be allowed

Venn Diagram from Previous Query



SELECT Example – Sorting Results with the ORDER BY Clause

- Sort the results first by STATE, and within a state by CUSTOMER_NAME

```
SELECT CUSTOMER_NAME, CITY, STATE
    FROM CUSTOMER_V
    WHERE STATE IN ('FL', 'TX', 'CA', 'HI')
    ORDER BY STATE, CUSTOMER_NAME;
```

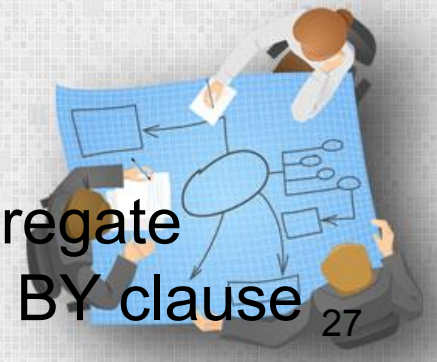
Note: the IN operator in this example allows you to include rows whose STATE value is either FL, TX, CA, or HI. It is more efficient than separate OR conditions

SELECT Example–
Categorizing Results Using the GROUP BY Clause

- For use with aggregate functions
  - ***Scalar aggregate***: single value returned from SQL query with aggregate function
  - ***Vector aggregate***: multiple values returned from SQL query with aggregate function (via GROUP BY)

```
SELECT CUSTOMER_STATE,
   COUNT(CUSTOMER_STATE)
     FROM CUSTOMER_V
     GROUP BY CUSTOMER_STATE;
```

Note: you can use single-value fields with aggregate functions if they are included in the GROUP BY clause



SELECT Example–
Qualifying Results by Categories Using the HAVING Clause

- For use with GROUP BY

```
SELECT CUSTOMER_STATE,
   COUNT(CUSTOMER_STATE)
     FROM CUSTOMER_V
     GROUP BY CUSTOMER_STATE
     HAVING COUNT(CUSTOMER_STATE) > 1;
```

Like a WHERE clause, but it operates on groups (categories), not on individual rows. Here, only those groups with total numbers greater than 1 will be included in final result

Using and Defining Views

- Views provide users controlled access to tables
- Base Table–table containing the raw data
- Dynamic View
  - A “virtual table” created dynamically upon request by a user
  - No data actually stored; instead data from base table made available to user
  - Based on SQL SELECT statement on base tables or other views
- Materialized View
  - Copy or replication of data
  - Data actually stored
  - Must be refreshed periodically to match the corresponding base tables

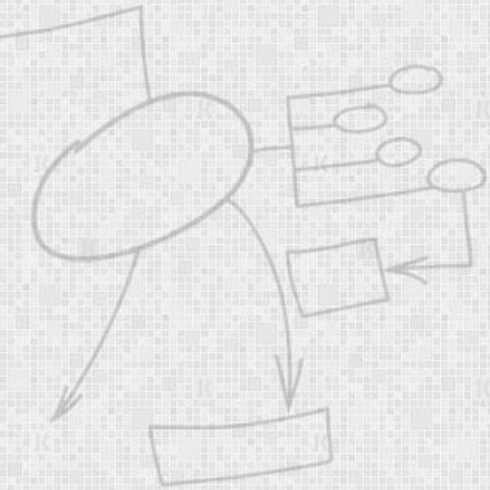
Sample CREATE VIEW

```
CREATE VIEW EXPENSIVE_STUFF_V AS
SELECT PRODUCT_ID, PRODUCT_NAME, UNIT_PRICE
FROM PRODUCT_T
WHERE UNIT_PRICE >300;
```

- View has a name
- View is based on a SELECT statement

Materialized Views Oracle

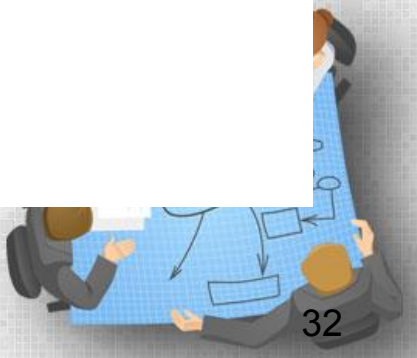
```
CREATE MATERIALIZED VIEW
mvemprowid
REFRESH WITH ROWID
AS SELECT * FROM emp;
```



Temporary Tables

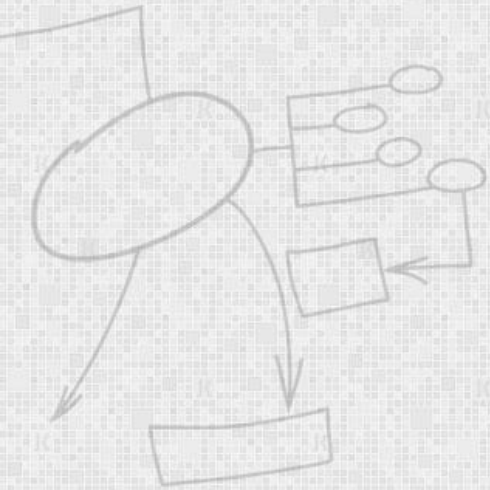
```
SELECT name, age, gender
INTO #MaleStudents
FROM student
WHERE gender = 'Male'
```

These are like materialized views but their life is limited



Indexes

Indexes are used to speed-up query, resulting in high performance



Clustered Indexes

- A clustered index defines the order in which data is physically stored in a table.
- Table data can be sorted in only way, therefore, there can be only one clustered index per table.
- In SQL Server, the primary key constraint automatically creates a clustered index on that particular column
- Custom Clustered index can be created after deleting primary key index

```
CREATE CLUSTERED INDEX Student_Name
ON student (Name)
```

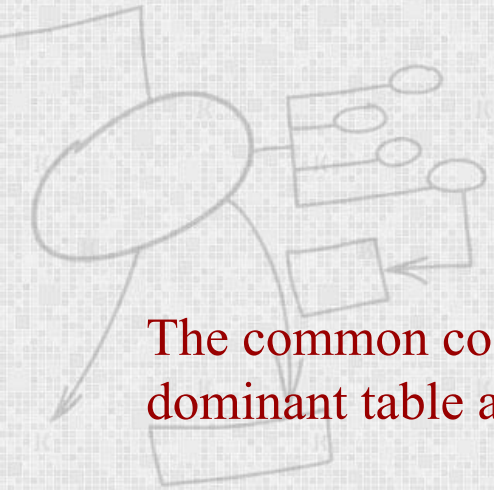
Non Clustered Indexes

- A non-clustered index doesn't sort the physical data inside the table.
- In fact, a non-clustered index is stored at one place and table data is stored in another place.
- This allows for more than one non-clustered index per table.
- Go to that row address and fetch other column values. It is due to this additional step that non-clustered indexes are slower than clustered indexes.

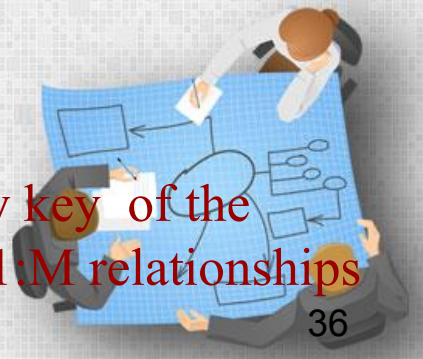
```
CREATE NONCLUSTERED INDEX Student_Name
ON student(name ASC)
```

Processing Multiple Tables–Joins

- Join–a relational operation that causes two or more tables with a common domain to be combined into a single table or view
- Equi Join



The common columns in joined tables are usually the primary key of the dominant table and the foreign key of the dependent table in 1:M relationships

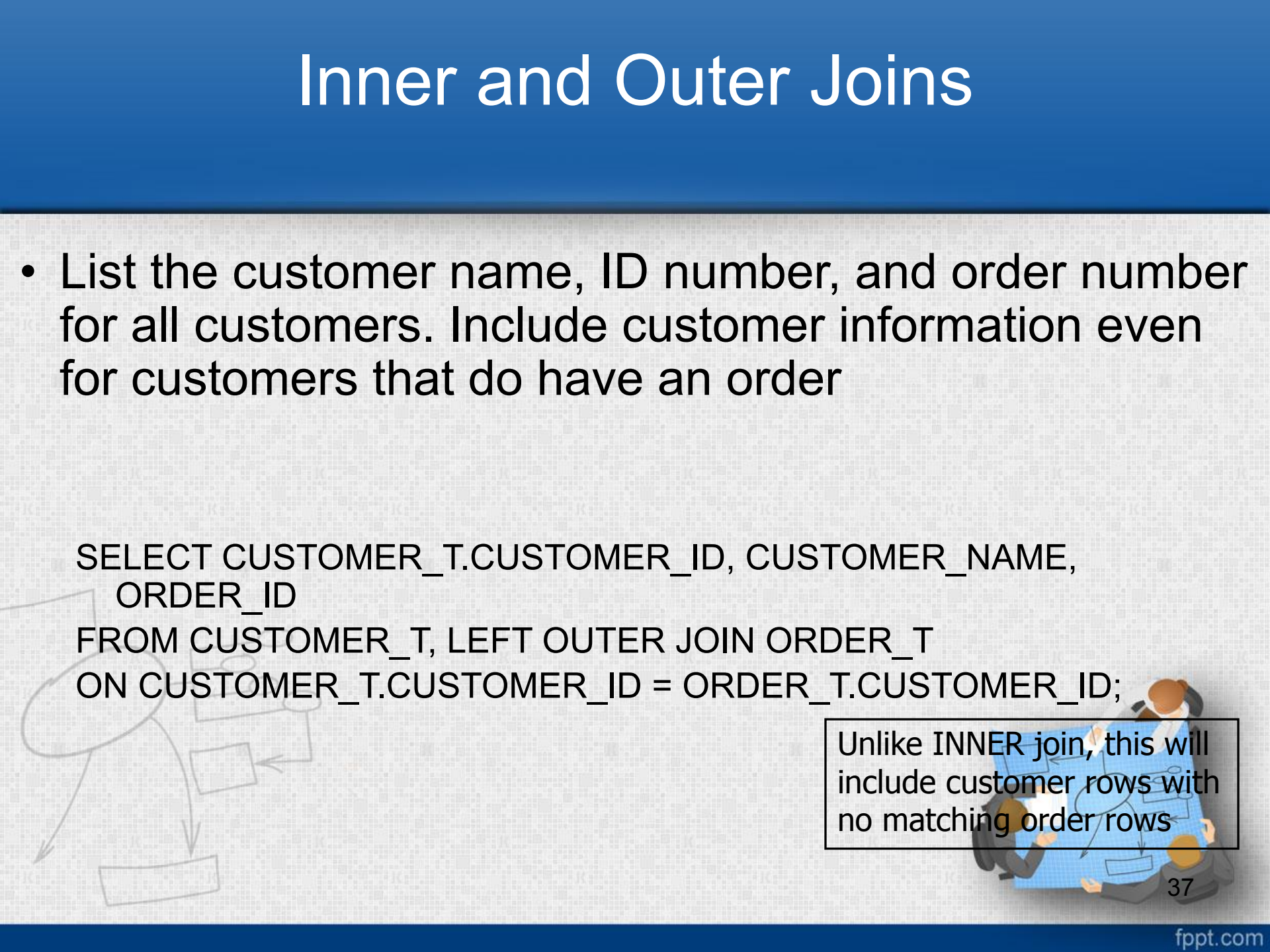


Inner and Outer Joins

- List the customer name, ID number, and order number for all customers. Include customer information even for customers that do have an order

```
SELECT CUSTOMER_T.CUSTOMER_ID, CUSTOMER_NAME,
   ORDER_ID
FROM CUSTOMER_T, LEFT OUTER JOIN ORDER_T
ON CUSTOMER_T.CUSTOMER_ID = ORDER_T.CUSTOMER_ID;
```

Unlike INNER join, this will include customer rows with no matching order rows



Results

Result:

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID
1	Contemporary Casuals	1001
1	Contemporary Casuals	1010
2	Value Furniture	1006
3	Home Furnishings	1005
4	Eastern Furniture	1009
5	Impressions	1004
6	Furniture Gallery	
7	Period Furnishings	
8	California Classics	1002
9	M & H Casual Furniture	
10	Seminole Interiors	
11	American Euro Lifestyles	1007
12	Battle Creek Furniture	1008
13	Heritage Furnishings	
14	Kaneohe Homes	
15	Mountain Scenes	1003

16 rows selected.

Unlike INNER join, this will include customer rows with no matching order rows

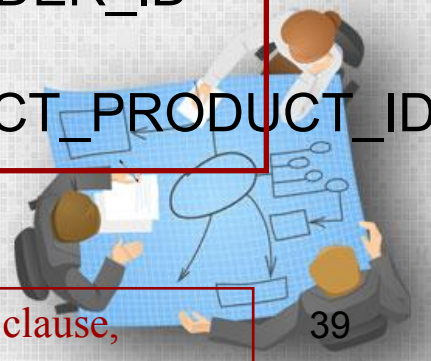
Multiple Table Join Example

- Assemble all information necessary to create an invoice for order number 1006

Four tables involved in this join

```
SELECT CUSTOMER_T.CUSTOMER_ID, CUSTOMER_NAME,
   CUSTOMER_ADDRESS, CITY, SATE, POSTAL_CODE,
   ORDER_T.ORDER_ID, ORDER_DATE, QUANTITY,
   PRODUCT_DESCRIPTION, STANDARD_PRICE,
   (QUANTITY * UNIT_PRICE)
FROM CUSTOMER_T, ORDER_T, ORDER_LINE_T, PRODUCT_T

WHERE  CUSTOMER_T.CUSTOMER_ID =
   ORDER_LINE.CUSTOMER_ID    AND ORDER_T.ORDER_ID =
   ORDER_LINE_T.ORDER_ID
         AND ORDER_LINE_T.PRODUCT_ID = PRODUCT_PRODUCT_ID
         AND ORDER_T.ORDER_ID = 1006;
```



Each pair of tables requires an equality-check condition in the WHERE clause, matching primary keys against foreign keys

Results from a four-table join

From CUSTOMER_T table

CUSTOMER_ID	CUSTOMER_NAME	CUSTOMER_ADDRESS	CUSTOMER_CITY	CUSTOMER_ST	POSTAL_CODE
2	Value Furniture	15145 S.W. 17th St.	Plano	TX	75094 7743
2	Value Furniture	15145 S.W. 17th St.	Plano	TX	75094 7743
2	Value Furniture	15145 S.W. 17th St.	Plano	TX	75094 7743

From ORDER_T table

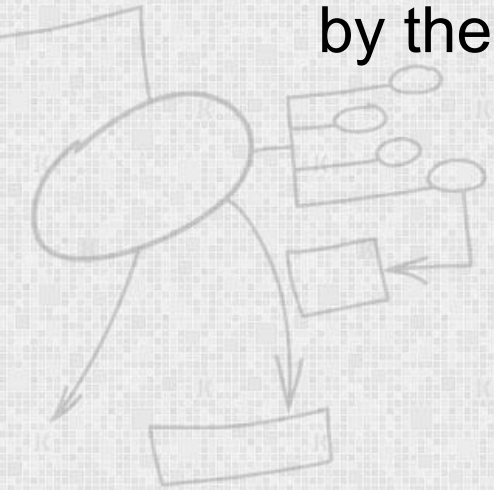
ORDER_ID	ORDER_DATE	ORDERED_QUANTITY
1006	24-OCT-06	1
1006	24-OCT-06	2
1006	24-OCT-06	2

From PRODUCT_T table

PRODUCT_NAME	STANDARD_PRICE	(QUANTITY* STANDARD_PRICE)
Entertainment Center	650	650
Writer's Desk	325	650
Dining Table	800	1600

Processing Multiple Tables Using Subqueries

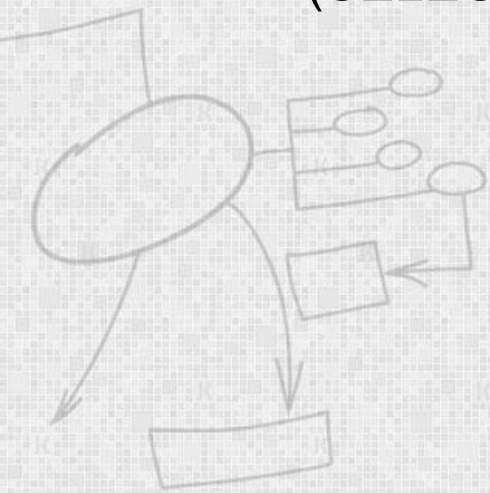
- Subquery–placing an inner query (SELECT statement) inside an outer query
- Subqueries can be:
  - Noncorrelated–executed once for the entire outer query
  - Correlated–executed once for each row returned by the outer query

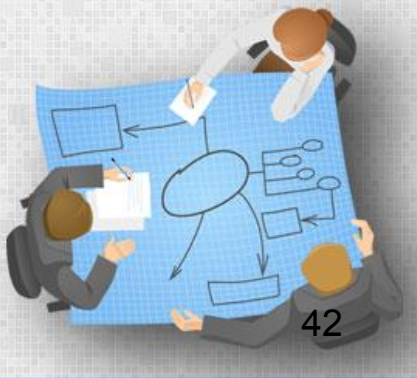


Subquery Example

- Show all customers who have placed an order

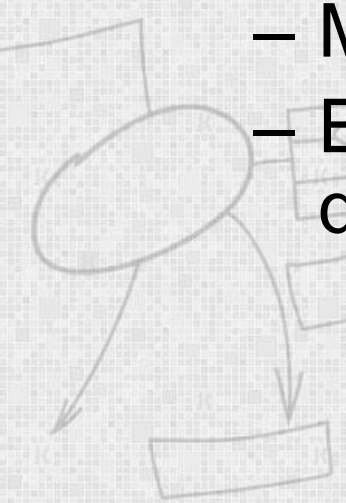
```
SELECT CUSTOMER_NAME     FROM CUSTOMER_T
WHERE CUSTOMER_ID  IN
          (SELECT DISTINCT CUSTOMER_ID FROM ORDER_T);
```





Correlated vs. Noncorrelated Subqueries

- Noncorrelated subqueries:
  - Do not depend on data from the outer query
  - Execute once for the entire outer query
- Correlated subqueries:
  - Make use of data from the outer query
  - Execute once for each row of the outer query



Union Queries

- Combine the output (union of multiple queries) together into a single result table

```
SELECT C1.CUSTOMER_ID,CUSTOMER_NAME,ORDERED_QUANTITY,
'Largest Quantity' QUANTITY
  FROM CUSTOMER_T C1,ORDER_T O1, ORDER_LINE_T Q1
        WHERE C1.CUSTOMER_ID =O1.CUSTOMER_ID
        AND O1.ORDER_ID =Q1.ORDER_ID
        AND ORDERED_QUANTITY =
              (SELECT MAX(ORDERED_QUANTITY)
              FROM ORDER_LINE_T)
```

First query

Combine →

```
UNION
SELECT C1.CUSTOMER_ID,CUSTOMER_NAME,ORDERED_QUANTITY,
'Smallest Quantity'
  FROM CUSTOMER_T C1,ORDER_T O1, ORDER_LINE_T Q1
        WHERE C1.CUSTOMER_ID =O1.CUSTOMER_ID
        AND O1.ORDER_ID =Q1.ORDER_ID
        AND ORDERED_QUANTITY =
              (SELECT MIN(ORDERED_QUANTITY)
              FROM ORDER_LINE_T)
ORDER BY ORDERED_QUANTITY;
```

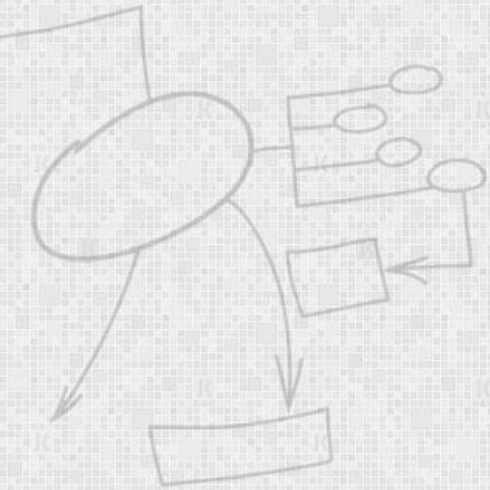
Second query

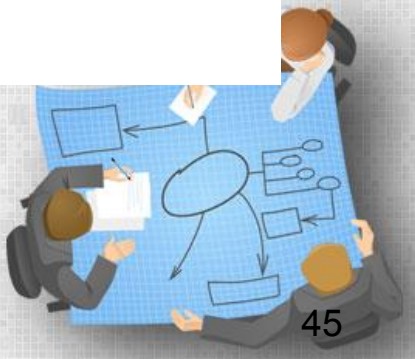
Conditional Expressions Using Case Syntax

- For conditional selection

```
{CASE expression
{WHEN expression
THEN {expression | NULL}} . . .
 | {WHEN predicate
THEN {expression | NULL}} . . .
[ELSE {expression  NULL}]
END }
 | ( NULLIF (expression, expression) }
 | ( COALESCE (expression . . .) }
```

Figure 8-4
CASE conditional syntax





Ensuring Transaction Integrity

- Transaction = A discrete unit of work that must be completely processed or not processed at all
  - May involve multiple updates
  - If any update fails, then all other updates must be cancelled
- SQL commands for transactions
  - BEGIN TRANSACTION/END TRANSACTION
    - Marks boundaries of a transaction
  - COMMIT
    - Makes all updates permanent
  - ROLLBACK
    - Cancels updates since the last COMMIT

An SQL Transaction sequence (in pseudocode)

```
BEGIN transaction

  INSERT Order_ID, Order_date, Customer_ID into Order_t;

  INSERT Order_ID, Product_ID, Quantity into Order_line_t;
  INSERT Order_ID, Product_ID, Quantity into Order_line_t;
  INSERT Order_ID, Product_ID, Quantity into Order_line_t;

END transaction
```

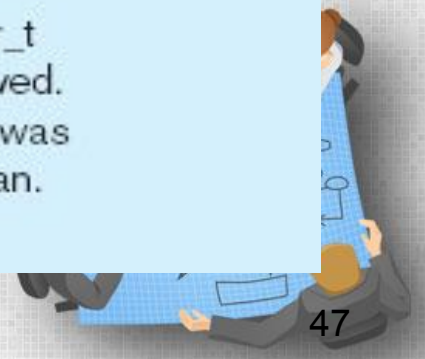
Valid information inserted.
COMMIT work

All changes to data
are made permanent.

Invalid Product_ID entered

Transaction will be ABORTED.
ROLLBACK all changes made to Order_t

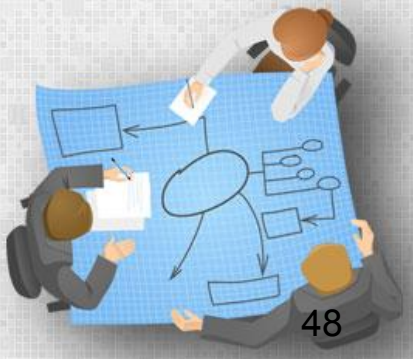
All changes made to Order_t
and Order_line_t are removed.
Database state is just as it was
before the transaction began.



Triggers

- A special kind of procedure which fires when an event occurs in a database
- We can execute a SQL query that will "do something" in a database when an event is fired

```
create trigger deep
on emp
for
insert,update,delete
as
print'you can not insert,update and delete this table i'
rollback;
```

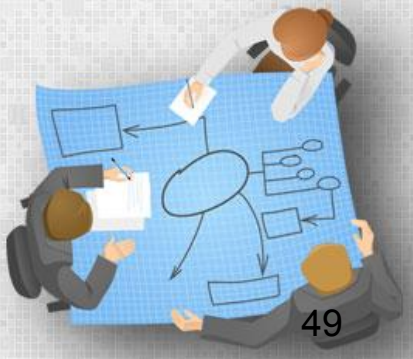


Stored Procedures

- A prepared SQL code that you can save, so the code can be reused over and over again

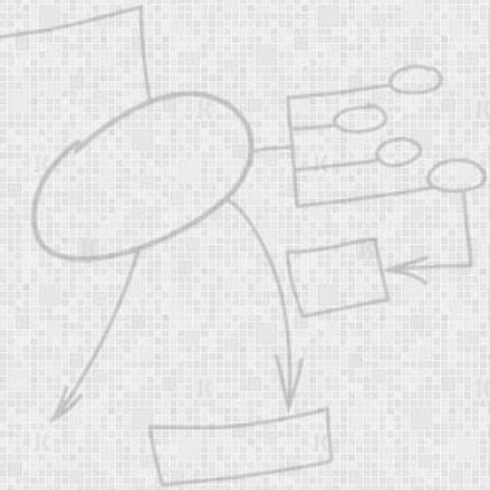
```
CREATE PROCEDURE procedure_name
AS
sql_statement;

EXEC procedure_name;
```



Stored Procedures

```
CREATE PROCEDURE SelectAllCustomers @City
nvarchar(30)
AS
SELECT * FROM Customers WHERE City = @City;
```



Stored Procedures Vs Triggers